Computing ACPR, modulated output power, and EVM from a 1-tone, swept harmonic balance simulation

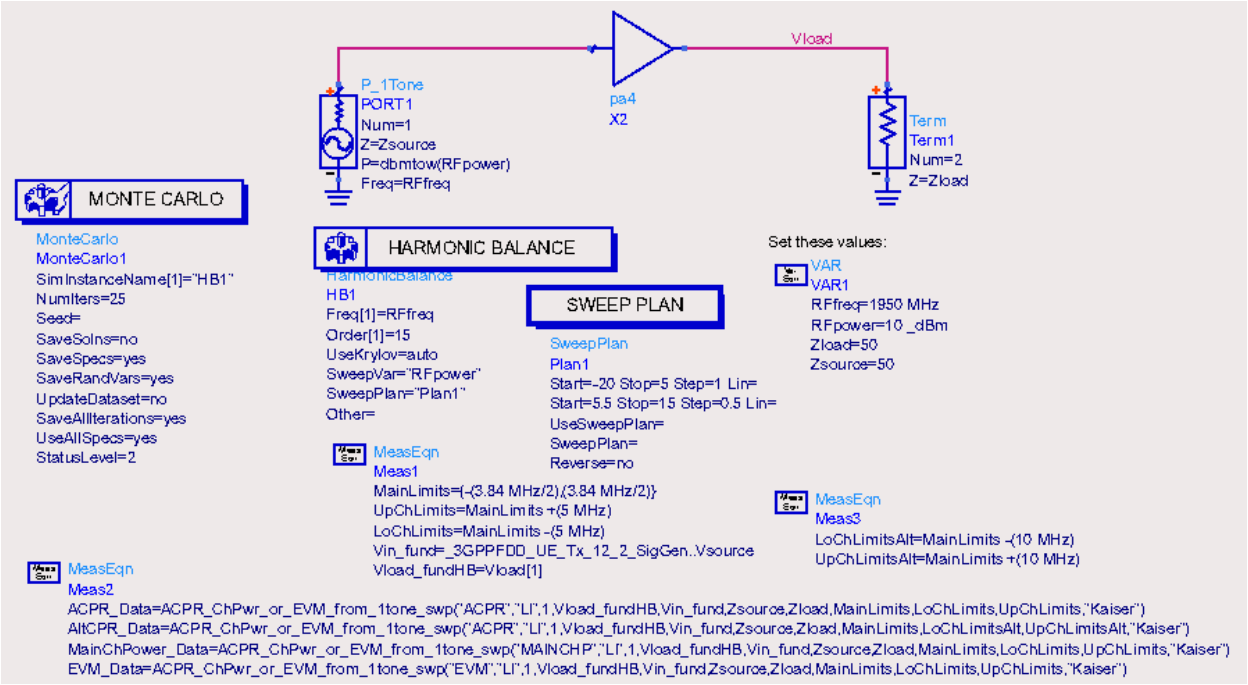
Andy Howard, 10/7/2010

This describes a fast way of computing ACPR, modulated output power, and/or EVM of an amplifier versus input power from a 1-tone, swept harmonic balance simulation. This is an outline of the steps (many of these are carried out by an AEL function):

1) Run a swept input power harmonic balance simulation of the amplifier.
2) Determine the amplitude-out-versus-amplitude-in (some would call this AM-to-AM) and phase-out-versus-phase-in (AM-to-PM) transfer functions of the amplifier from the harmonic balance simulation.
3) Obtain (from a separate simulation or from some file) the envelope-versus-time waveform of the modulated input signal. This should be the magnitude and phase of the signal.
4) Apply this envelope-versus-time signal to the transfer functions from step 2 above to compute the envelope-versus-time of the output signal.
5) Use the fs() function to compute the spectrum of the modulated output signal.
6) From this spectrum, compute the powers in the main, adjacent, and alternate channels. Compute the adjacent- and alternate-channel power ratios from these values.
7) Compute EVM as follows. First compute the mean phase and RMS amplitude differences between the input and output modulated signals. At each time point, the ideal modulated output signal magnitude and phase should be the input modulated signal shifted by the mean phase difference and scaled by the RMS amplitude difference. The magnitude of the distance between the modulated output signal and this ideal modulated output signal is the EVM at each time point. The computed EVM is the RMS value of all the EVM values versus time.

These various calculations can be done while including an extra sweep, for example, of a parameter value or with a Monte Carlo analysis. The calculations are carried out using special measurement expressions that have been added to ADS 2011 that you may place on the schematic or in the data display.

Here is an example applying the functions to compute ACPR, main channel power, and EVM:

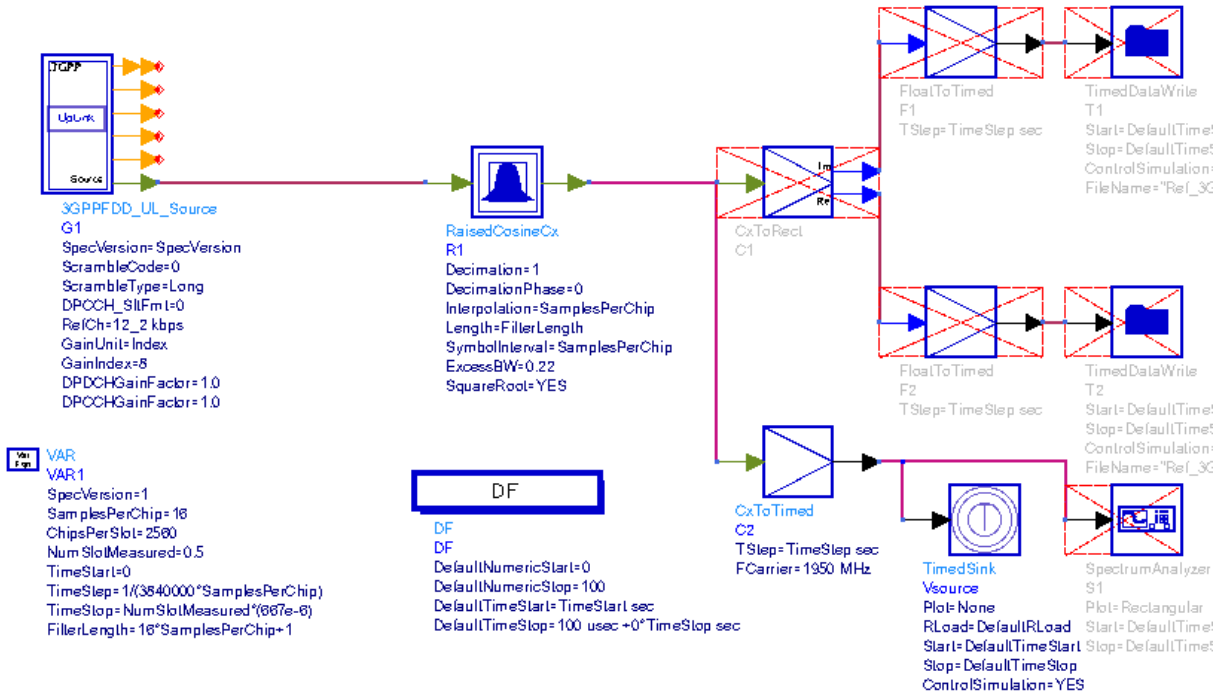


The “pa4” subcircuit is a power amplifier. A Monte Carlo simulation is run, and for each trial a swept-power harmonic balance simulation is run. For higher accuracy, we use a finer step size as the input power is increased driving the amplifier into compression. For the ACPR and main channel power calculations, the **MainLimits** equation sets the main channel bandwidth at +/-1.92 MHz. The **UpChLimits** equation sets the upper adjacent channel frequency limits to the same bandwidth as the main channel but offset by +5 MHz. The **LoChLimits** equation is similar. The **LoChLimitsAlt** and **UpChLimitsAlt** equations define the lower and upper alternate channel limits, respectively.

The **Vin_fund** equation reads in the envelope of the modulated signal from a dataset that was generated from an ADS example, examples/WCDMA3G/WCDMA3G_SignalSource_prj/3GPPFDD_UE_Tx_12_2_SigGen schematic, with modifications as shown:

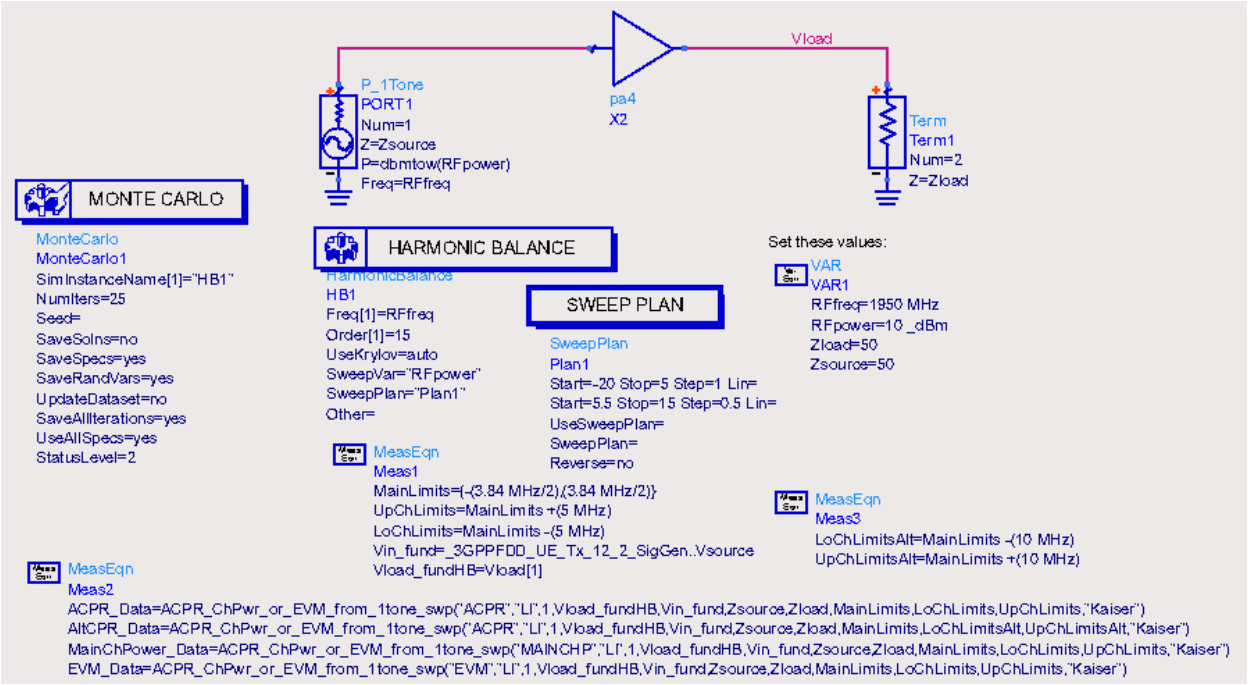
3GPP FDD: Generate the UE 12.2kbps data

1. The design generates the file-based signal source.
2. This signal source can be used for other measurments, such as spurious emission tests.
3. Signal spectrum is shown by 3GPPFDD_UE_Tx_12_2.dds
4. Simulation time is about 1 minute.



The stop time was reduced from 100 to 50 usec., which gives 3073 time points. This is a sufficiently long time record to give reasonable results. The greater the number of time points in the modulated data file, the longer the time required to compute ACPR, etc. However, it is possible to use a subset of the data. (In the **ACPR_ChPwr_or_EVM_from_1tone_swp()** function, use **Vin_fund[0::1000]** instead of **Vin_fund** to use just the first 1001 time points, for example.)

Returning to the harmonic balance setup:



the **Vload_fundHB=Vload[1]** equation specifies the complex fundamental output voltage which will be used in the ACPR_ChPwr_or_EVM_from_1tone_swp() function for the computation of the magnitude and phase transfer functions.

What the ACPR_ChPwr_or_EVM_from_1tone_swp() functions do

The **ACPR_ChPwr_or_EVM_from_1tone_swp(returnVal, algorithm, allowextrap, charVoltage, inputSig, sourceZ, loadZ, mainCh, lowerAdjCh, upperAdjCh, winType, winConst)** function computes the ACPR values (upper and lower), the main channel power, or the EVM. These are the passed parameters:

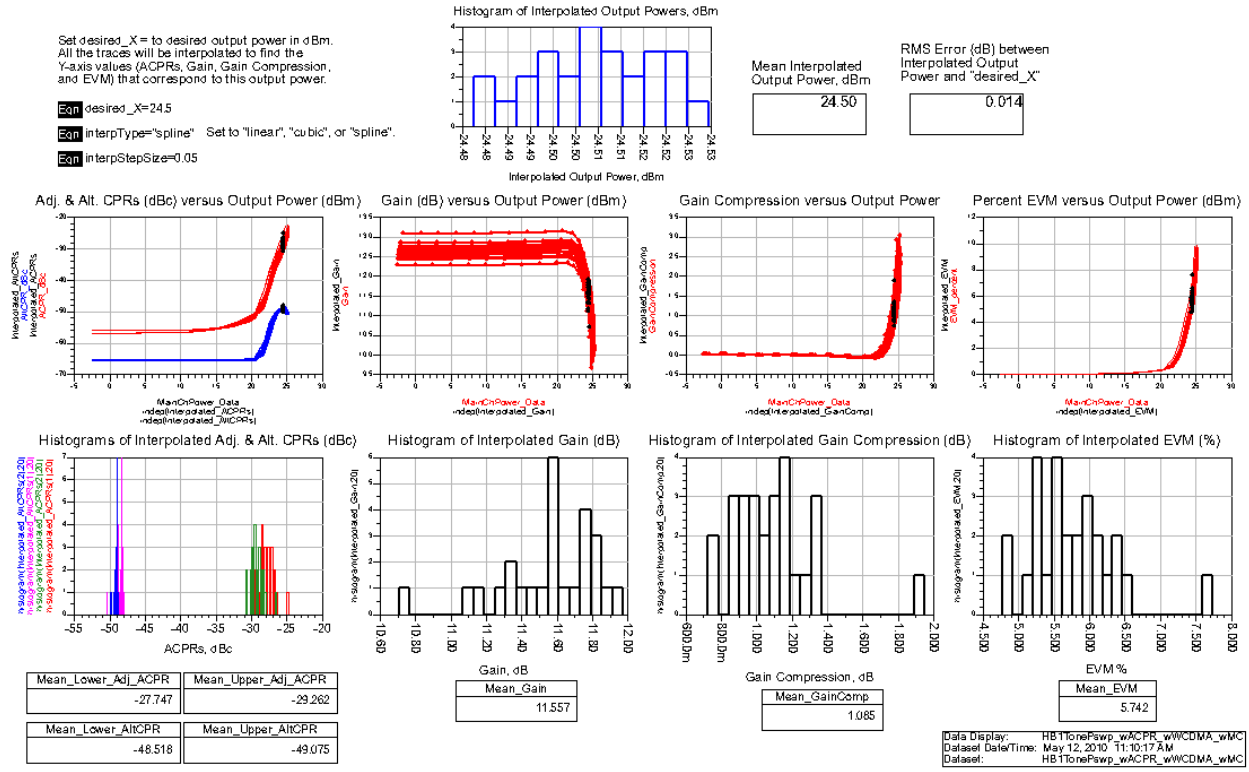
1) **returnVal** –flag for defining what to return. **"ACPR"** returns the ACPR, **"MAINCHP"** returns the main channel power, and **"EVM"** returns the error vector magnitude.
2) **algorithm** –flag for setting the algorithm for approximating the vout/vin transfer function. Set to **"CF"** for curve fit or **"LI"** for linear interpolation. Our experience has been that linear interpolation produces better results, although it takes longer.
3) **allowextrap** –flag for allowing or disallowing extrapolation. If set to 1, scale factors will be used to vary the power of the modulated input signal between the maximum input power from the harmonic balance sweep and this maximum power – 30 dB. If this flag is set to 0 then any scale factor that, when applied to the modulated input signal, would cause its peak value to exceed the maximum input power of the harmonic balance sweep, will not be allowed.
4) **charVoltage** –this is the characterization voltage (the fundamental output voltage from the harmonic balance sweep.) Example: **Vload_fund,** where **Vload_fund=Vload[1].**
5) **inputSig** –this is the input modulated signal (the envelope.) This signal should just be a function of time. See screen capture above for an example.

6) **sourceZ** – this is the source impedance.
7) **loadZ** –this is the load impedance.
8) **mainCh** –these are the main channel frequency limits, as an offset from the carrier frequency. Example: **{(-3.84 MHz/2),(3.84 MHz/2)}**
9) **lowerAdjCh** –these are the lower adjacent (or alternate) channel frequency limits as an offset from the carrier frequency. Example: **MainLimits – (5 MHz)**
10) **upperAdjCh** –these are the upper adjacent (or alternate) channel frequency limits as an offset from the carrier frequency.
11) **winType** –window type (for example, **"Kaiser"**). Possible windows are exactly the same as those used in the **fs()** function.
12) **winConst –**window constant. Possible constant values are associated with each window type and are exactly the same as the values used in the **fs()** function. Leave blank to use the default value.

Here is the sequence of steps carried out by the **ACPR_ChPwr_or_EVM_from_1tone_swp()** function:

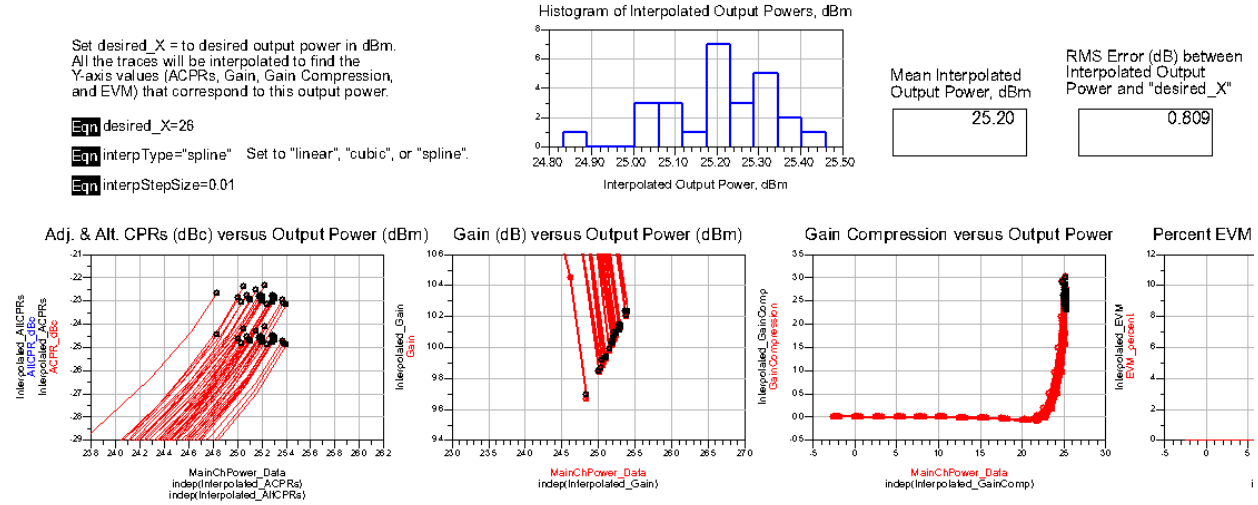
1) Compute the scale factors based on the maximum input power from the swept harmonic balance simulation (call this MaxInPwrHB.) These scale factors are hard-coded such that the average power of the modulated input signal will be scaled from MaxInPwrHB -30 dB to MaxInPwrHB -7.5 dB in 2.5 dB steps and from MaxInPwrHB -6 dB to MaxInPwrHB in 1 dB steps. This assumes extrapolation is allowed. If extrapolation is not allowed, then some of these higher scale factors may not be used.
2) Compute the Vout/Vin transfer function using either a curve fit or linear interpolation (preferred.)
3) Scale the input modulated signal using the scale factors from step 1 above.
4) Apply the scaled input signal (now a function of the scale factor and time) to the transfer function to get the output voltage versus both scale factor and time.
5) If **returnVal** is **"ACPR"** then return the ACPR versus the modulated input signal power.
6) If **returnVal** is **"MAINCHP"** then return the main channel power versus the modulated input signal power.
7) If **returnVal** is **"EVM"** then return the EVM (this is the raw EVM, not specification-compliant) versus the modulated input power.

This data display shows the computed results:



This shows Adjacent and Alternate channel power ratios versus output power, gain versus output power, gain compression versus output power, and EVM versus output power. You enter the desired output power value, **desired_X,** interpolation is carried out, and the histograms show the distribution of each response while the amplifier is delivering output power equal (approximately) to **desired_X**.

If you specify a **desired_X** that is beyond the x-axis range of the data, then just the maximum x-axis values of the traces will be used. This may result in a significant difference between the desired output power (**desired_X**) and the mean interpolated output power:



These are the equations that carry out these calculations:

```
Eqn ACPR_dBc=ACPR_Data

Eqn Pout_dBm=MainChPower_Data

Eqn ACPR_vs_Pout=vs(ACPR_dBc,Pout_dBm)

Eqn Interpolated_ACPRs=interpolate_swept_data(ACPR_vs_Pout,desired_X,interpStepSize,interpType)

Eqn Mean_Lower_Adj_ACPR=mean(Interpolated_ACPRs(1))
Eqn Mean_Upper_Adj_ACPR=mean(Interpolated_ACPRs(2))

Eqn AltCPR_dBc=AltCPR_Data
Eqn AltCPR_vs_Pout=vs(AltCPR_dBc,Pout_dBm)
Eqn Interpolated_AltCPRs=interpolate_swept_data(AltCPR_vs_Pout,desired_X,interpStepSize,interpType)

Eqn Mean_Lower_AltCPR=mean(Interpolated_AltCPRs(1))
Eqn Mean_Upper_AltCPR=mean(Interpolated_AltCPRs(2))

Eqn Gain=Pout_dBm-indep(Pout_dBm)

Eqn Interpolated_Gain=interpolate_swept_data(vs(Gain,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_Gain=mean(Interpolated_Gain)

Eqn GainCompression=if (sweep_dim(Gain)>1) then expand(Gain[0])-Gain else Gain[0]-Gain
Eqn Interpolated_GainComp=interpolate_swept_data(vs(GainCompression,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_GainComp=mean(Interpolated_GainComp)

Eqn EVM_percent=EVM_Data
Eqn Interpolated_EVM=interpolate_swept_data(vs(EVM_percent,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_EVM=mean(Interpolated_EVM)

Eqn Interpolated_Pout_dBm=permute(indep(Interpolated_ACPRs))

Eqn RMS_Error=sqrt(sum((Interpolated_Pout_dBm-desired_X)**2)/sweep_size(Interpolated_Pout_dBm))
```

In this case, **ACPR_Data, MainChPower_Data, AltCPR_Data,** and **EVM_Data** were all calculated on the schematic using the **ACPR_ChPwr_or_EVM_from_1tone_swp()** function.

The **interpolate_swept_data(original_data_Y_vs_X, desired_X, interpStepSize, interpType)** function is another new function that has been added to ADS 2011. It returns the interpolated Y value that corresponds to (and is versus) the interpolated X value, which will be closest to **desired_X**. This result is returned as a single value or an array, depending on the dimensionality of the **original_data_Y_vs_X** argument. These are the passed parameters:

1) **original_data_Y_vs_X** – any 1-, 2-, or 3-dimensional set of curves of Y versus X data.
2) **desired_X –** desired X value, to be found by interpolation.
3) **interpStepSize** – the smaller this number is, the closer the interpolation will get to the desired x value.
4) **interpType** – **“linear”**, **“cubic”** or **“spline”**. This specifies the type of interpolation.

This function does not do any extrapolation. If the requested **desired_X** is above or below the maximum or minimum X value, then it just returns the Y value corresponding to the maximum or minimum X value, respectively.

Using equations on the data display to compute the ACPRs, main channel power, or EVM

There is another custom function, **Mod_Data_from_1tone_swpUNI()** that returns the adjacent and alternate channel powers, main channel power, and EVM. Because this function returns multiple results, it cannot be used in a measurement expression on the schematic. It can only be used in the data display. The advantage of using this function instead of **ACPR_ChPwr_or_EVM_from_1tone_swp()** is that it is more efficient for obtaining all these results. When using **ACPR_ChPwr_or_EVM_from_1tone_swp()**, you have to call it once to get the adjacent channel power ratios, once again to get the alternate channel power ratios, once again to get the main channel power, and once again to get the EVM. The big disadvantage of using the **Mod_Data_from_1tone_swpUNI()** is that this function will get executed each time you open a data display that contains it. While it is slower to use the **ACPR_ChPwr_or_EVM_from_1tone_swp()** function on the schematic, the advantage is that the results are written into the dataset, and data displays that show these results open instantly.

The **Mod_Data_from_1tone_swpUNI(algorithm, allowextrap, charVoltage, inputSig, sourceZ, loadZ, mainCh, mainChForPout, lowerAdjCh, upperAdjCh, lowerAltCh, upperAltCh, winType, winConst)** function computes the ACPR values (upper and lower), Alternate CPR values (upper and lower), the main channel power, and the EVM. These are the passed parameters:

1) **algorithm –**flag for setting the algorithm for approximating the vout/vin transfer function. Set to **"CF"** for curve fit or **"LI"** for linear interpolation. Our experience has been that linear interpolation produces better results, although it takes longer.
2) **allowextrap** –flag for allowing or disallowing extrapolation. If set to 1, scale factors will be used to vary the power of the modulated input signal between the maximum input power from the harmonic balance sweep and this maximum power – 30 dB. If this flag is set to 0 then any scale factor that, when applied to the modulated input signal, would cause its peak value to exceed the maximum input power of the harmonic balance sweep, will not be allowed.
3) **charVoltage** –this is the characterization voltage (the fundamental output voltage from the harmonic balance sweep.) Example: **Vload_fund, where Vload_fund=Vload[1].**
4) **inputSig** –this is the input modulated signal (the envelope.) This signal should just be a function of time. See screen capture above for an example.
5) **sourceZ** – this is the source impedance.
6) **loadZ** –this is the load impedance.
7) **mainCh** –these are the main channel frequency limits, as an offset from the carrier frequency. Example: **{(-3.84 MHz/2),(3.84 MHz/2)}**
8) **mainChForPout** – these are the frequency limits used for computing the modulated output power. Normally this would be the same as the **mainCh** argument, but this allows you to specify a different bandwidth for computing the modulated output power.

9) **lowerAdjCh** –these are the lower adjacent channel frequency limits as an offset from the carrier frequency. Example: **MainLimits – (5 MHz)**
10) **upperAdjCh** –these are the upper adjacent channel frequency limits as an offset from the carrier frequency.
11) **lowerAltCh** –these are the lower alternate channel frequency limits as an offset from the carrier frequency. Example: **MainLimits – (10 MHz)**
12) **upperAltCh** –these are the upper alternate channel frequency limits as an offset from the carrier frequency. Example: **MainLimits + (10 MHz)**
13) **winType** –window type (for example, **"Kaiser"**). Possible windows are exactly the same as those used in the **fs()** function.
14) **winConst –**window constant. Possible constant values are associated with each window type and are exactly the same as the values used in the **fs()** function. Leave blank to use the default value.

This set of data display equations uses the **Mod_Data_from_1tone_swpUNI()** function.

```
Eqn MainLimitsForPout=MainLimits

Eqn Data_DDS=Mod_Data_from_1tone_swpUNI("LI",1,Vload_fundHB,Vin_fund,Zsource[0],Zload[0],MainLimits,MainLimitsForPout,LoChLimits,UpChLimits,LoChLimitsAlt,UpChLimitsAlt,"Kaiser")

Eqn ACPR_dBc=Data_DDS(0)
Eqn Pout_dBm=Data_DDS(2)
Eqn ACPR_vs_Pout=vs(ACPR_dBc,Pout_dBm)
Eqn Interpolated_ACPRs=interpolate_swept_data(ACPR_vs_Pout,desired_X,interpStepSize,interpType)
Eqn Mean_Lower_Adj_ACPR=mean(Interpolated_ACPRs(1))
Eqn Mean_Upper_Adj_ACPR=mean(Interpolated_ACPRs(2))

Eqn AltCPR_dBc=Data_DDS(1)
Eqn AltCPR_vs_Pout=vs(AltCPR_dBc,Pout_dBm)
Eqn Interpolated_AltCPRs=interpolate_swept_data(AltCPR_vs_Pout,desired_X,interpStepSize,interpType)
Eqn Mean_Lower_AltCPR=mean(Interpolated_AltCPRs(1))
Eqn Mean_Upper_AltCPR=mean(Interpolated_AltCPRs(2))

Eqn Gain=Pout_dBm-indep(Pout_dBm)
Eqn Interpolated_Gain=interpolate_swept_data(vs(Gain,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_Gain=mean(Interpolated_Gain)
Eqn GainCompression=if (sweep_dim(Gain)>1) then expand(Gain[0])-Gain else Gain[0]-Gain
Eqn Interpolated_GainComp=interpolate_swept_data(vs(GainCompression,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_GainComp=mean(Interpolated_GainComp)

Eqn EVM_percent=Data_DDS(3)
Eqn Interpolated_EVM=interpolate_swept_data(vs(EVM_percent,Pout_dBm),desired_X,interpStepSize,interpType)
Eqn Mean_EVM=mean(Interpolated_EVM)
```

The only differences in these equations relative to those shown earlier are:

Data_DDS=Mod_Data_from_1tone_swpUNI(…)
ACPR_dBc=Data_DDS(0)
Pout_dBm=Data_DDS(2)
AltCPR_dBc=Data_DDS(1)
EVM_percent=Data_DDS(3)

The plots are all exactly the same.

Benchmark test results:

A simulation of a much more complex amplifier with these custom equations just on the data display

took about 4 minutes for 10 MC trials (+1 for the nominal case) and about 1 minute and 31 seconds to open the data display. This was using a modulated input signal with 3073 time points.

The same simulation with 4 equations, each using the **ACPR_ChPwr_or_EVM_from_1tone_swp()** function to compute the adjacent channel power ratios, alternate channel power ratios, main channel power, and EVM took about 3 minutes 54 seconds to run the simulation and another 4 minutes and 45 seconds to evaluate the equations.

Technical discussion comparing this 1-tone swept power harmonic balance simulation technique for computing ACPR and EVM and the Ptolemy co-simulation approach

For EVM, the single-tone method is not specification-compliant. It just measures the "raw" EVM, computed at each time point. The EVM is computed after correcting for the average phase difference and RMS amplitude difference between the output and input modulated signals. If the modulated signal at the output of the amplifier has only a constant phase shift and a constant gain (meaning that neither vary with the amplitude of the input modulated signal), then the EVM will be zero. With this method, the EVM is computed at each time point, not at just the symbol times. There is no demodulation or decoding of the signal, so you can't calculate the EVM of each sub-carrier, say for an LTE signal.

With the Ptolemy method, the EVM calculation uses an EVM sink that is specification-compliant.

The only advantage of the single-tone method is that it may be faster. Clearly it is not as accurate, but might still be useful as a proxy for the specification-compliant EVM.

For ACPR, the single-tone method does not include any receive-side filtering. It just generates the spectrum at the output of the amplifier, integrates the power in the main, adjacent, and alternate channels, then computes the ratios.

The single-tone method of computing EVM (and ACPR) will tend to become less accurate as the bandwidth of the signal gets larger. This is because this method assumes the response of the amplifier is constant across the modulation bandwidth (we're modeling the nonlinearity by injecting a single tone at the carrier frequency, after all.)

With Ptolemy fast co-simulation, the behavioral model that is created can include frequency variation across the modulation bandwidth.